

Module 4 Guided Lab Session 1: Request Flow & Visibility

Field	Value
Module	M4 ASP.NET Core 10 Fundamentals
Exercises	1 (Middleware Ordering), 1B (Custom Request Logging Middleware)
After this session you can show	Anonymous calls get 401 on a protected TMS route; every response carries X-Correlation-Id ; logs tie request/response to that id

Welcome to the Backend Foundation Sprint

You keep building the **same** Training Management System (TMS) API you will extend it. A previous build's Program.cs **runs**, but behaviour on a sensitive route is wrong: you will discover what is wrong by **observing HTTP status and tracing the pipeline**, then fix it. Later you add tracing so failures are tied to a single request.

In M1 you modelled Student, Course, Enrollment, Assessment, and related ideas. In M4 you host those concerns on the web server. The least goal by this lab is to make a sensitive read behave like a protected API in HTTP, then add **request logging + correlation ID** so support and debugging are grounded in evidence.

This session focuses on request pipeline behavior and middleware ordering. You are not implementing a production authentication system in Session 1. The goal is to observe how ASP.NET Core behaves when a protected endpoint participates in the authentication and authorization pipeline.

Later sessions will introduce more complete API functionality and authentication scenarios.

By the end of Exercise 1: GET /api/assessments/results rejects anonymous callers with **401** (no assessment JSON for unauthenticated requests). You earn that outcome by fixing how the pipeline is wired **without** relying on comments inside the starter file to name the mistake for you.

By the end of Exercise 1B: you wrap the pipeline in custom logging, emit **X-Correlation-Id**, and register UseExceptionHandler early the same posture you will reconnect when Session 3 adds ProblemDetails and Scalar.

Before you begin: the TmsApi project

Module 4 uses **one** ASP.NET Core Web API project named **TmsApi**. Create it **once** at the start of Session 1 and keep working in that same folder through Session 3 so exercises stack on each other. If you closed your machine and are **continuing**, open the existing TmsApi directory you already created do **not** run dotnet new again for a second API.

Open your terminal and run these commands **one time** when you begin:

Check your SDK version: must show 10.x

```
dotnet --version
```

Create the Web API project (controller-based template; matches later exercises)

```
dotnet new webapi -n TmsApi --no-openapi --use-controllers
```

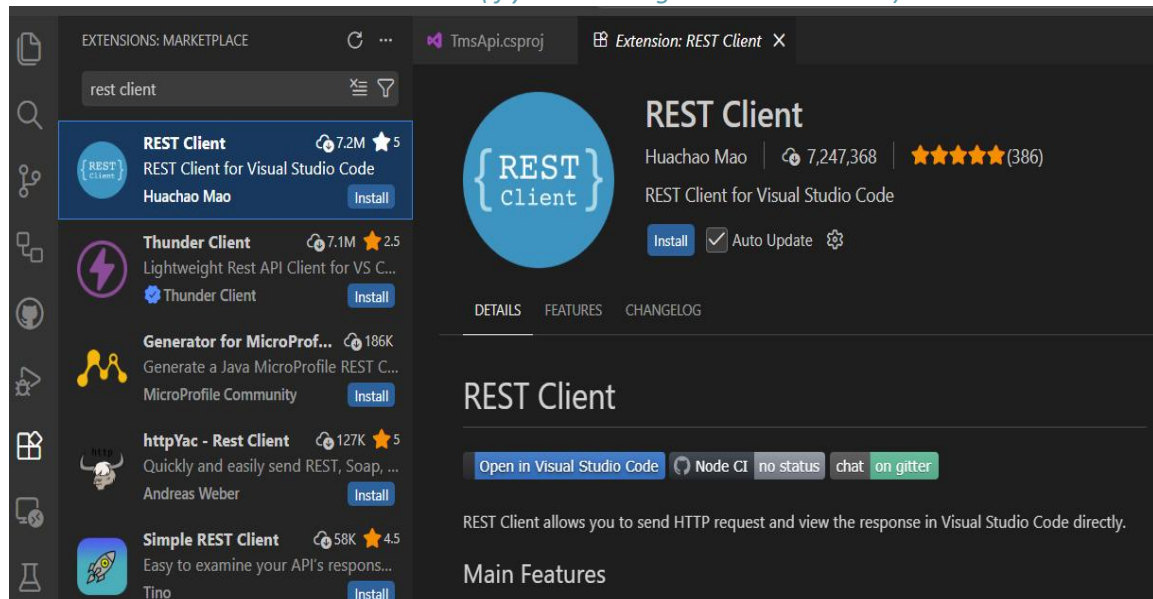
Move into the project directory

```
cd TmsApi
```

Open it in VS Code (or Visual Studio)

code .

Add REST Client extension in VS Code (if you are using Visual Studio code)



```
dotnet run
```

You should see output like:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5xxx
```

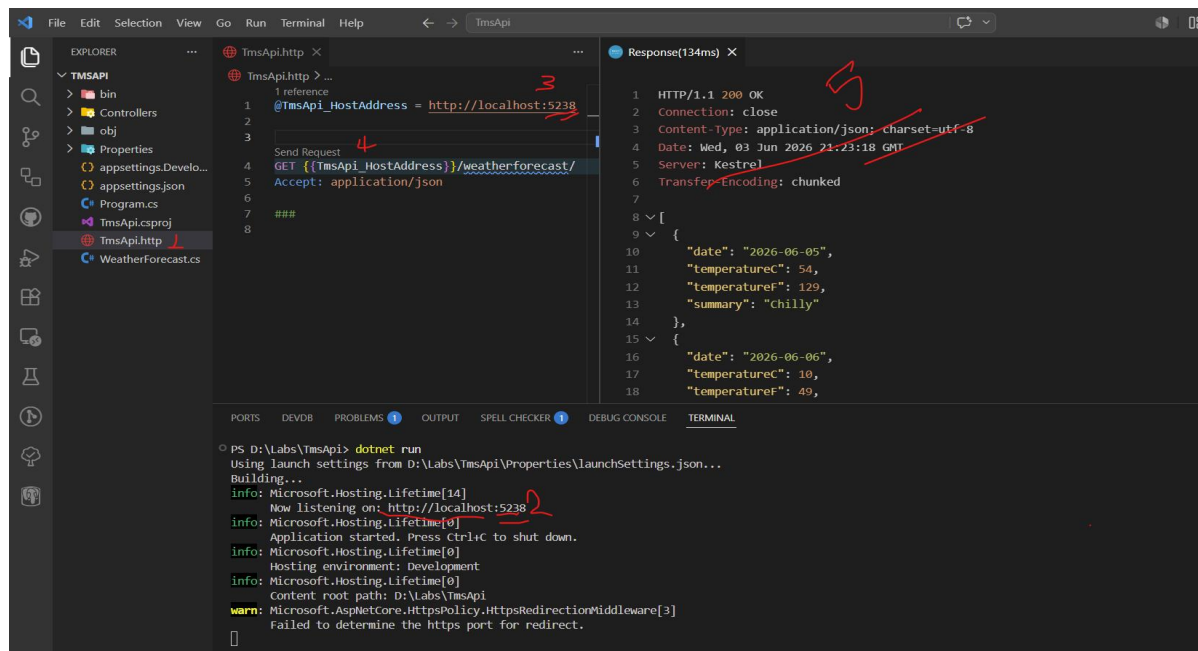
Press Ctrl+C to stop the server.

Troubleshooting

Problem	Likely cause	Fix
dotnet --version shows 8.x or 9.x	Wrong SDK	Install .NET 10 from dot.net
Port already in use	Another process on 5000	dotnet run --urls "http://localhost:5050"
Template produces minimal API not controllers	Default changed in some SDK builds	Always pass --use-controllers as shown above
dotnet: command not found	SDK not on PATH	Reopen terminal after install; or fully qualify the path

Testing HTTP

After running the api and installing the extension, we can test the endpoints from the [TmsApi.http](#) file. Open it and click on the send request button, you will see the response on the right side from headers to body all included.



Every URL example below uses `http://localhost:5000`; replace it with the port your terminal prints.

Exercise 1: The Blind Server (Middleware Ordering)

Context: Assessment outcomes must stay confidential. The route GET `/api/assessments/results` returns a **placeholder** JSON body for now (you will replace it with real persistence and DTOs in later modules). Below is **starter** `Program.cs` from a prior check-in. Run it, hit the route anonymously, then read the file **top to bottom** and decide what should happen for a protected read versus what actually happens.

Predict before you change code

Trace one incoming GET in order: which `app.Use...` / `Map...` lines run, and in what sequence? Where does the response get committed? Write your prediction in one sentence, then run the app and compare.

```
// Starter pipeline (do not assume this order is correct)
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseRouting();

app.MapGet("/api/assessments/results", () => Results.Ok(new
{
    courseCode = "CS-101",
    studentId = "S-001",
    letterGrade = "A"
}));

app.UseAuthentication();
app.UseAuthorization();

app.Run();
```

Reminder: Services and Middleware

Many ASP.NET Core features require two separate steps:

1. Register the required services during application startup.
2. Add the corresponding middleware to the request pipeline.

When troubleshooting, verify both steps. Middleware often depends on services that were registered earlier in **`Program.cs`**.

Your task: secure the pipeline

Exercise Assumption

For this exercise, focus on protecting the endpoint and placing authentication and authorization correctly in the request pipeline.

You are not required to implement a production authentication mechanism.

The objective is to observe the difference between:

Unprotected endpoint

→ 200 OK

Protected endpoint

→ Request challenged before endpoint execution

The route should require authorization and participate in the authentication/authorization pipeline.

Rewrite Program.cs so GET /api/assessments/results is **not** anonymous: unauthenticated callers get **401**, not the JSON body. You keep the same placeholder response for callers who **are** allowed through.

Work from your template's usual Program.cs shape; **do not** paste a full solution from elsewhere unless your facilitator green-lights it. Use this scaffold and fill the TODOs from your understanding of the pipeline:

```
var builder = WebApplication.CreateBuilder(args);  
// Services: add authentication / authorization services
```

```
var app = builder.Build();
```

```
// TODO 1: Register routing in the pipeline where it belongs for your app.  
// TODO 2: Register authentication and authorization in the pipeline where your template and fa  
cilitator expect them for a protected minimal API route.  
// TODO 3: Map GET /api/assessments/results with the same response body as the starter, but r  
equire authorization for that route.
```

```
app.Run();
```

Run / verify / common failure

- **Run:** dotnet run
- **Check:** Browser GET http://localhost:5000/api/assessments/results with **no** login or bearer token.
- **Expected:** 401 Unauthorized not 200 with JSON, not 404.

Why 401?

A protected endpoint should not execute for anonymous callers.

When authentication and authorization participate correctly in the request pipeline, ASP.NET Core prevents the endpoint from running and returns 401 Unauthorized instead.

If you still receive the JSON response body, trace the request path through the pipeline again and verify that the endpoint is actually protected.

- **Common failure 404:** Terminal middleware or branch ordering re-read the middleware / request-pipeline section in **Module 4 ASP.NET Core Fundamentals** (your Essentials reading).
- **Common failure still 200 with body on anonymous GET:** Middleware that should run for this route is not in the active pipeline path trace order again or ask for a hint in the room.

Note:

In custom middleware, `RequestDelegate next` represents the remainder of the request pipeline.

Calling: `await next(context);` passes control to the next middleware or endpoint.

When downstream processing completes, execution returns to the current middleware so that additional work (such as logging) can be performed.

Middleware Reference Card

The following APIs are commonly used inside custom middleware.

```
// Read the request path
var path = context.Request.Path;

// Read the HTTP method
var method = context.Request.Method;

// Add a response header
context.Response.Headers["X-Correlation-Id"] = correlationId;

// Read the response status code
var statusCode = context.Response.StatusCode;

// Measure elapsed time
var stopwatch = Stopwatch.StartNew();

// work happens here
stopwatch.Stop();

var elapsedMs = stopwatch.ElapsedMilliseconds;
```

```
// Write a structured log entry
_logger.LogInformation(
    "Request {Method} {Path}",
    context.Request.Method,
    context.Request.Path);

// Pass control to the next middleware
await _next(context);
```

Typical middleware flow

```
// Before next
// - Read request information
// - Add response headers
// - Start timing

await _next(context);

// After next
// - Read response information
// - Stop timing
// - Write completion logs
```

Exercise 1B: Custom Request Logging Middleware

Context: Order is fixed, but you still cannot tie log lines to a single failing request. Implement middleware that logs start/finish and stamps **X-Correlation-Id**.

Your Task (implement, do not transcribe a full solution)

1. Add RequestLoggingMiddleware.cs with a RequestDelegate next and ILogger<RequestLoggingMiddleware>.
2. In InvokeAsync:
 - Generate a short correlation id (for example from Guid.NewGuid().ToString("N")[..8]).
 - Set context.Response.Headers["X-Correlation-Id"] **before** await next(context).
 - Use Stopwatch to measure elapsed time.
 - Log one line on entry (method, path, correlation id) and one on exit (status code, elapsed ms, same id).
3. In Program.cs, register **in this order**

- `app.UseMiddleware<RequestLoggingMiddleware>();` first (outer wrapper).
- Then `UseExceptionHandler`, `UseHttpsRedirection`, `UseRouting`, `UseAuthentication`, `UseAuthorization`.
- Map `GET /api/assessments/results` last, still with `.RequireAuthorization()`.

If you are unsure where `UseExceptionHandler` fits, use **Module 4 ASP.NET Core Fundamentals** (middleware and error-handling sections) you are wiring the slot now so later exercises can plug in `ProblemDetails` without reordering everything.

Run / verify / common failure

- **Run:** `dotnet run`
- **Check:** Same anonymous GET as Exercise 1 (`/api/assessments/results`).
- **Expected:** Response still **401**; response headers include **X-Correlation-Id**; **two** log lines per request sharing the same correlation id.
- **Common failure no logs:** Middleware registered after the endpoint.
- **Common failure no header:** Header set after `await next(context)` (too late).

Why the Header Must Be Added Early?

Response headers should normally be added before:

`await next(context);`

A downstream middleware or endpoint may begin sending the response. Once the response has started, additional headers may no longer be applied.

Setting **X-Correlation-Id** before calling `next` ensures the identifier is available on successful responses as well as error responses.

Why this session ends here

You have only done two things and that is enough for Session 1:

1. **Exercise 1:** A **TMS assessment** route is **not** reachable anonymously (401 without auth); the pipeline matches what you learned in the Essentials reading on request flow.
2. **Exercise 1B:** Every request is **observable**: correlation id in the response and paired log lines. You also **registered** `UseExceptionHandler` in the stack you will extend when error bodies become `ProblemDetails`.

Looking Ahead

Looking Ahead

This session validates the unauthorized path only:

Anonymous caller → 401 Unauthorized

Later modules will introduce authentication mechanisms that allow you to verify both outcomes:

Anonymous caller → 401 Unauthorized

Authenticated caller → 200 OK

For this now, the objective is to understand middleware ordering, request flow, and request observability.

Session 1 checkpoint

Before Session 2, confirm:

- ☐ Anonymous GET /api/assessments/results returns **401**, not 200 with JSON.
- ☐ Every response includes **X-Correlation-Id** (DevTools Network headers, or Scalar response headers after Session 3).
- ☐ Console shows **two** structured lines per request with the **same** correlation id.
- ☐ Optional: when /scalar/v1 exists, you can repeat the GET in **Try it** and see the same status and header without a curl wall.

If any check fails, finish Session 1 before moving on.

Lab Support Code: Training Authentication Handler

To keep Session 1 focused on middleware ordering rather than production authentication, use the following temporary authentication handler.

Create a file named `TrainingAuthHandler.cs`.

```
using System.Security.Claims;
using System.Text.Encodings.Web;
using Microsoft.AspNetCore.Authentication;
using Microsoft.Extensions.Options;

public class TrainingAuthHandler
    : AuthenticationHandler<AuthenticationSchemeOptions>
{
    public TrainingAuthHandler(
        IOptionsMonitor<AuthenticationSchemeOptions> options,
        ILoggerFactory logger,
        UrlEncoder encoder)
        : base(options, logger, encoder)
    {
    }

    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        if (!Request.Headers.ContainsKey("X-Training-User"))
        {
            return Task.FromResult(

```

```

        AuthenticateResult.Fail("Missing training user header."));
    }

    var claims = new[]
    {
        new Claim(ClaimTypes.Name, Request.Headers["X-Training-User"]!)
    };

    var identity = new ClaimsIdentity(claims, Scheme.Name);
    var principal = new ClaimsPrincipal(identity);
    var ticket = new AuthenticationTicket(principal, Scheme.Name);

    return Task.FromResult(
        AuthenticateResult.Success(ticket));
}
}

```

This handler treats requests containing an `X-Training-User` header as authenticated. Requests without that header are treated as unauthenticated.

The handler exists only for training purposes and will be replaced by real authentication approaches in later modules.

Register the Training Authentication Scheme

Register authentication and authorization services during application startup.

```
builder.Services
```

```

    .AddAuthentication("Training")
    .AddScheme<AuthenticationSchemeOptions,

```

```
TrainingAuthHandler>("Training", null);
```

```
builder.Services.AddAuthorization();
```

Configure the middleware pipeline:

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

Protect the endpoint:

```
app.MapGet(...)
```

```
    .RequireAuthorization();
```